

Jinping Zhang

Lead iOS Engineer

Email: zjinping28@gmail.com ; Phone: (929) 466-4268

Profile Summary

Lead iOS Developer with 10+ years of experience building dynamic, user-focused applications, including **6 apps published on the App Store**.

- Solid experience in **Banking/Finance, Ecommerce, Airlines, Insurance, Healthcare (regulated industries, enterprise scale), Automotive/EV**.
- Highly skilled in Swift and Objective-C, with broad expertise in iOS frameworks such as SwiftUI, Combine, MapKit, UIKit, Core Graphics, Core Animation, Core Bluetooth, Core Data, and CloudKit. Experienced in applying MVVM, MVC, MVVM-C, VIPER and Clean Architecture to deliver clean, scalable codebases.
- Strong background in Agile and Scrum development, actively contributing to sprint planning and execution. Well-versed in TDD and CI/CD practices, using design patterns like Singleton, Observer, and Delegation to solve complex problems. Proficient with asynchronous networking using URLSession, Async Await, Actor, GCD, Alamofire, and AFNetworking, and experienced with Combine and RxSwift for reactive, responsive applications.
- Skilled in creating intuitive user interfaces with SwiftUI, UIKit, Storyboards, and XIBs, while ensuring inclusivity through Apple's Accessibility APIs. Hands-on experience with Jenkins, Fastlane, Azure DevOps, Bitrise, CircleCI Hudson, XCTest, Instruments, LLDB, and NSLogs for CI, testing, and debugging. Comfortable with source control systems, including Git, Stash, SourceTree, and SVN.
- In-depth knowledge of Core ML, RealityKit, and ARKit for building immersive AR experiences and integrating machine learning into apps. Experienced in using Firebase Realtime Database and Firebase Cloud Messaging for data sync and push notifications. Recognized for maintaining high coding standards, writing clear documentation, and delivering reliable, production-ready software.
- Strong communicator and mentor, with a proven ability to collaborate with stakeholders, designers, product managers, and cross-functional teams. Known for adopting emerging iOS technologies early to enhance efficiency, introduce innovative features, and continuously improve product quality.
- Leveraged AI-powered tools like GitHub Copilot, Cursor and GitHub Actions to accelerate code generation, enforce coding standards, and automate testing workflows, improving development efficiency and reducing delivery timelines

Technical Skills

- **Programming Languages:** Swift, Objective-C, C++, JavaScript, Python, Kotlin, Dart, HTML5/CSS3 SQL, MySQL
- **Cross Platform Framework:** React Native, Flutter, Kotlin Multi-Platform KMM
- **SwiftUI Components:** Property Wrappers, NavigationStack, View Modifiers, View Builders, Button Styles, Group Box Styles, Label Styles, Menu Styles
- **Architecture & Design Patterns:** MVC, MVP, MVVM, VIPER, Clean Architecture, Observer Pattern, Delegation, Decorator, Factory, Builder, Singleton, Visitor, Proxy, Interpreter, Command
- **UI/UX:** SwiftUI, Storyboards, AutoLayout, Interface Builder, XIBs, Programmatic UI creation
- **Web Services:** REST, GraphQL, SOAP, WebSockets REST-JSON, SOAP-XML, Reachability, URLSession, Decoder, NSJSONSerialization, NSXMLSerialization, Apple Push Notification (APN), UserDefaults, NotificationCenter, OperationQueue, MessageUI, Codable, JSONDecoder
- **Frameworks/Libraries:** SwiftUI, CoreData, Combine, SwiftData, Swift Testing.Cocoa Touch, Cocoa Pods, Core Audio, CloudKit, UIKit, RealityKit & ARKit, RxSwift, RxCocoa, Core Animation, Fabric, Crashlytics, CorePlot, MapKit, Metal, Core Bluetooth, Reachability, RxAlamoFire, SwiftyJSON, Swinject, Realm, React Js, React Native, Apollo Client. AWS
- **Continuous Integration:** Jenkins, Fastlane, GitLab, Bitrise, Github | **CI/CD:** Jenkins, Fastlane, Azure CI, Azure Boards, Carthage, SPM
- **iOS Tools:** CocoaTouch, Cocoapods, Storyboards, AutoLayout, Instruments, XIBs, MVC, MVVM, VIPER, Visual Paradigm, Interface Builder, Size Classes, UIKit, Core Graphics, CALayers, UIView, Views, View Controllers, MapKit, Core Location
- **Cocoa Touch Framework:** App Extension, Handoff, Document Picker, AirDrop, TextKit, UIKit Dynamics, Multitasking, UI, State Preservation, Apple Push Notification Service, Local Notifications, Gesture Recognizers, Standard System View Controllers
- **IDE & Tools:** XCode, JIRA, Confluence, SourceTree, Git, GitHub, SVN, VS Code
- **Database, Data Persistence:** Core Data, SwiftData, SQLite, Realm, Firebase
- **Project Methods:** Agile, Scrum, Test Driven Development, Pair Programming

- **Quality Testing and Troubleshooting:** Grand Central Dispatch (GCD), Instruments, Allocations, Leaks, Zombies, LLDB, Unit Tests, Functional Testing, Test Flight, Charles Proxy, Jest, Cypress, Detox
- **Machine Learning:** Create ML, ML APIs, Vision, MLKit | **Services:** Apple Push Notification Services (APNS)
- **Threading:** Swift Concurrency, Async await, Actor Grand Central Dispatch (GCD), NSOperations | **Media:** AV Foundation, AVKit
- **Streaming Technologies:** Adaptive Bitrate Streaming, Video Compression, Bitrate
- **Security:** SSL pinning, biometric authentication, AES encryption, OAuth 2.0, Azure AD, Keychain, CryptoKit
- **Miscellaneous:** Async Await, CLEAN, MVVM-C, State, ObservedObject, Table/TableView, NFC, Localization, SDWebImage, Map Annotation, SelectAnnotationView, ViewForAnnotation, NSLog, Breakpoints
- **Testing Tools:** BrowserStack, XC UI Tests, Danger | **Analytics:** Adobe Analytics, Firebase Analytics

Professional Experience

Lead iOS Engineer

Mar 2024 – Present

M&T Bank, New York, NY

Link: <https://apps.apple.com/us/app/m-t-mobile-banking/id397761931>

Summary: As Lead iOS Engineer for the M&T Bank Mobile App, I developed core banking features including account summary, mobile check deposit, bill pay, Zelle transfers, and card management. I modernized legacy modules with SwiftUI + UIKit integration, leveraging Combine and async/await for responsive user experiences. Collaborated with the Android team for cross-platform feature parity and mentored engineers in SwiftUI and modular architecture best practices.

- Developed key banking workflows, including Account Summary, Transaction History, and Mobile Check Deposit using SwiftUI + Combine, improving app responsiveness and reducing UI-related crash rates by 20%.
- Rebuilt Bill Pay and Zelle P2P Transfer flows with SwiftUI forms, property wrappers (@State, @StateObject, @ObservedObject, @EnvironmentObject), streamlining state management and boosting transaction success rates by 15%.
- Designed a SwiftUI Dashboard for Budget Tracking and Cash Flow Insights, integrating Charts framework for interactive financial visualizations, which increased feature adoption by 18%.
- Integrated Apple Wallet boarding for debit/credit cards and Card Management workflows (lock/unlock, PIN reset) with SwiftUI navigation stacks, reducing customer support calls by 12%.
- Built the Mobile Check Deposit module using AVFoundation for check image capture and validation, achieving a 98% same-day deposit success rate.
- Implemented real-time Alerts & Notifications module with SwiftUI List + async/await networking, allowing customizable fraud/security alerts and driving a 30% increase in alert engagement.
- Refactored legacy UIKit modules (Statements, Transfers) into SwiftUI components, modularized via Swift Packages, improving maintainability and enabling faster feature delivery.
- Applied protocol-oriented programming, Dependency Injection, and Coordinator pattern to manage flows and screen transitions (e.g., enrolments, card actions), improving the app's navigation consistency.
- Ensured data security with 256-bit SSL encryption, multi-factor authentication during login (user ID/password + OTP), and idle session logout to adhere to banking grade security standards.
- Strengthened security with OAuth2.0, Keychain storage, and end-to-end encryption, reducing unauthorized login attempts to near zero.
- Expanded automated testing (XCTest + XCUITest) for login, transfers, and deposits, achieving 85% coverage and minimizing regression defects across bi-weekly releases.
- Enhanced CI/CD pipelines with Fastlane + Jenkins + Xcode Cloud, automating builds, UI tests, and TestFlight distribution, reducing release cycles by 25%.
- Developed Mobile Check Deposit, Zelle P2P Transfers, and Bill Pay modules for the Android app using Kotlin, Coroutines, and Flow, ensuring seamless functionality and feature parity with the iOS (SwiftUI/Combine) implementation.
- Built the Budget & Cash Flow dashboards in Jetpack Compose, aligning visual consistency with iOS SwiftUI Charts, and implemented reactive state handling through ViewModel + Flow.
- Redesigned transaction history and activity feed using Jetpack Compose with lazy lists, improving scrolling performance and maintaining UI consistency with iOS SwiftUI equivalents.

- Engineered offline-first storage for boarding passes and card management, leveraging Room database with local encryption, ensuring reliable access and synchronization with backend services under intermittent connectivity.
- Led a 6-engineer iOS team through the migration from UIKit → SwiftUI, modularizing features via Swift Packages, and coaching on async/await + Combine best practices, reducing crash rates and speeding up delivery.
- Mentored both iOS and Android developers on cross-platform mobile banking domain knowledge, ensuring synchronized feature releases and smoother collaboration between platform teams.

Tech stack: Swift, SwiftUI, Combine, async/await, UIKit, Kotlin, Jetpack Compose, Coroutines, Flow, MVVM-C, Swift Packages, REST & GraphQL APIs, URLSession, Retrofit, Core Data, Room DB, Keychain, EncryptedSharedPreferences, OAuth 2.0, SSL Pinning, Apple Wallet, AVFoundation, XCTest, XCUITest, Espresso, Fastlane, Jenkins, Xcode Cloud, Firebase Crashlytics, SonarQube, SwiftLint, GitHub Actions.

Sr. iOS Engineer
Adidas | Portland, OR

Nov 2021 – Feb 2024

Link: <https://apps.apple.com/us/app/adidas-shop-shoes-clothing/id1266591536>

Summary: As Sr. iOS Engineer for the Adidas mobile app, I drove the modernization of the global e-commerce experience through a scalable MVVM + Coordinator architecture built with Swift, UIKit, and SwiftUI. Delivered key features such as personalized product recommendations using CoreML, AR-based Virtual Try-On experiences with ARKit, and a GraphQL + Apollo integration for efficient data fetching. Focused on performance optimization, security compliance (SSL pinning, biometric auth), and accessibility alignment with WCAG standards.

- Led development of the Adidas iOS app, enabling personalized shopping, order tracking, and exclusive product drops for over 5M+ global users, while maintaining brand consistency and premium performance.
- Architected a modular MVVM + Coordinator architecture using Swift + UIKit, improving code reusability, navigation flow, and app stability across multiple releases.
- Home & Product Feed: Rebuilt dynamic feed screens using SwiftUI + Combine integrated with UIKit containers. Used async/await network calls and @Published state management for real-time catalog refreshes and pagination.
- Product Detail Screen: Designed hybrid SwiftUI + ARKit product visualization, implementing CoreML-based Similar Product suggestions and async CoreML model inference for real-time results.
- AR Try-On Experience: Built an ARKit + SceneKit-based virtual try-on module using async rendering tasks, enabling realistic 3D placement and lighting adjustments for shoes in the user's environment.
- Checkout Flow: Implemented SwiftUI checkout and payment modules with async/await transaction handling, Apple Pay integration, and SSL-pinned endpoints for secure PCI-DSS compliant processing.
- Profile & Loyalty Dashboard: Built a SwiftUI dashboard using StateObject + EnvironmentObject, synchronizing rewards and points with Combine pipelines from GraphQL subscriptions for reactive updates.
- Reusable SwiftUI Components: Created a library of custom SwiftUI components (buttons, modals, loaders, and product cards) using ViewBuilder + Generics, reducing UI duplication across 15+ screens.
- Search & Filter: Optimized GraphQL search using Combine debounce operators and async/await API handling, improving search responsiveness by 35% and reducing redundant network calls.
- Cart & Wishlist: Used Core Data with Combine publishers and async background tasks, enabling offline persistence.
- Authentication & Onboarding: Created a modern SwiftUI login and onboarding flow using OAuth2.0 + async/await token handling, biometric login via LocalAuthentication, and session refresh via Combine streams.
- Push Notifications & Deep Links: Handled deep link routing in SwiftUI views using Combine subjects for event propagation for marketing activities.
- AR Try-On Feature: Delivered an immersive SwiftUI-integrated ARKit try-on module, leveraging async SceneKit rendering and real-time lighting adjustments for realistic product visualization.
- Performance Optimization: Introduced async image loading and NSCache-backed prefetching with Combine, resulting in a 30% improvement in UI rendering and scroll fluidity.

- **Accessibility & Localization:** Ensured full WCAG 2.1 compliance across SwiftUI screens with Dynamic Type, VoiceOver, and async localized content loading for multi-region support.
- **Testing & CI/CD:** Achieved 85% unit test coverage with async XCTest and reactive state tests; integrated Fastlane + Azure Dev Ops pipelines for continuous testing, signing, and deployment.
- **Partnered with Android (Kotlin + Jetpack Compose) and backend (GraphQL) teams** to define shared async schema contracts, ensuring consistent architecture and data flow across platforms.
- **Leadership:** Mentored 4 mid-level iOS engineers on SwiftUI best practices, Combine debugging, and async/await migration, accelerating feature delivery while improving team technical depth.

Tech stack: Swift, SwiftUI, UIKit, Combine, Async Await, MVVM C, State, ObservedObject, ARKit, GraphQL, Apollo, RESTful APIs, URLSession, Codable, XCTest, Instruments, Crash Logs, Core Data, SSL pinning, biometric authentication, AES encryption, HostingViewController.

Sr iOS Developer Allegiant Air | Las Vegas, NV

Nov 2019 – Oct 2021

Link: <https://apps.apple.com/us/app/allegiant/id528413490>

Summary: As a Sr iOS Developer on the Allegiant Air mobile team, I helped modernize the passenger experience across booking, check-in, and trip management modules. The project focused on migrating a legacy UIKit + RxSwift codebase to a more reactive, maintainable architecture with Combine and async/await for the network layer. Led key refactors that reduced complexity, improved performance, and ensured a smooth transition to SwiftUI-based modules introduced in later phases.

- **UIKit Modernization:** Maintained and enhanced major UIKit-driven modules like Flight Search, Booking Flow, and Trip Summary, adopting MVVM-C architecture and dependency injection to improve modularity and testability.
- **RxSwift to Combine Migration:** Initiated a gradual migration from RxSwift to Combine, simplifying reactive data handling and reducing external dependency footprint across booking and payment flows.
- **Network Layer Refactor:** Rebuilt the legacy closure-based networking layer into a cleaner async/await-compatible structure using URLSession + Result types, improving readability and reducing callback complexity.
- **Seat Map & Add-Ons:** Delivered a real-time, interactive Seat Selection module using UICollectionViewCompositionalLayout and Combine, offering fluid animations, instant price recalculations, and seamless add-on updates.
- **Check-In & Boarding Pass:** Enhanced the Check-In experience with persistent Core Data caching and Apple Wallet boarding pass integration, ensuring access to passes offline.
- **Flight Status & Alerts:** Delivered a real-time Flight Status dashboard using Combine streams and background refresh tasks, providing live updates on gate changes and baggage claim status.
- **Authentication & Security:** Implemented OAuth 2.0 with token refresh logic and Keychain Services for credential storage, combined with Local Authentication (Face ID / Touch ID) for secure, password less logins.
- **Offline Access:** Developed offline-first capabilities using Core Data for saved trips, boarding passes, and loyalty points, improving usability during travel with limited connectivity.
- **Performance Optimization:** Leveraged Instruments to identify memory leaks and optimize scrolling performance in flight search results, reducing CPU usage by 20%.
- **Crash & Log Monitoring:** Integrated Firebase Crashlytics and Sentry for proactive crash analysis and performance trend tracking, maintaining a 99.7% crash-free session rate post-release.
- **Combine Adoption in New Modules:** Introduced Combine-based pipelines for Loyalty Dashboard and Trip History, improving state propagation and simplifying UI updates.
- **Gradual SwiftUI Adoption:** Introduced SwiftUI for isolated views like Boarding Pass Details and Trip Summary Cards, integrating them via UIHostingController within UIKit-based containers.
- **Testing & CI/CD:** Strengthened quality assurance through XCTest, XCUITest, and CI integration with Fastlane + Bitrise, automating builds and regression testing for faster, safer releases.
- **Team Mentorship:** Mentored mid-level engineers on Combine debugging, async/await adoption, and best practices for hybrid UIKit-SwiftUI architectures, improving code consistency team-wide.

- Collaborated with UI/UX teams, backend developers, and QA engineers in an Agile environment with daily scrums, sprint planning, and retrospectives.
- Implemented continuous integration pipelines using Jenkins, enabling frequent and stable production releases.
- Delivered detailed architectural designs for core app features while ensuring scalability, performance, and adherence to best practices.

React Native (Cross-Platform Internal Project):

Summary: Developed an internal employee productivity and support app using React Native, enabling teams to manage shift schedules, submit support tickets, and view analytics dashboards across iOS and Android platforms.

- **Core Screens:** Built key modules such as Dashboard (KPIs & Announcements), Task Manager (create/update workflows), Support Chat, and Profile & Settings, integrating them with Redux-Saga + REST APIs for consistent state synchronization and offline access.
- **Cross-Platform Architecture:** Designed and implemented a shared React Native architecture for iOS and Android, consolidating common business logic and UI components using JavaScript + Redux Toolkit, which reduced duplicate code by 40%.
- **Native Module Bridging:** Integrated native iOS (Swift) and Android (Kotlin) modules into React Native for device-level features such as camera access, biometric authentication, and offline storage, ensuring smooth hybrid integration.
- **Performance Optimization:** Enhanced rendering performance by implementing React.memo, FlatList virtualization, and lazy loading, improving frame rates and reducing UI lag on complex list screens.
- **State Management & Networking:** Built a reactive state management system using Redux-Saga + Axios for API orchestration, handling retries, caching, and error boundaries for reliable offline/online synchronization.
- **Continuous Integration & Delivery:** Streamlined release management with Fastlane + Firebase App Distribution, enabling automated build pipelines and beta deployments for cross-functional testing teams.

Tech Stacks: Swift 5.3 → 5.5, UIKit, SwiftUI (introduced gradually with iOS 14), Combine, RxSwift (legacy), MVVM-C, Core Data, WalletKit, Keychain, OAuth 2.0, Apple Pay, URLSession, REST APIs, GraphQL (Apollo), XCTest/XCTest, Fastlane, Jenkins, Firebase, Crashlytics, Sentry, Instruments, JIRA, Confluence, React Native (0.64 → 0.70), JavaScript, Redux Toolkit, Redux-Saga, Axios, React Navigation v6, React Query (for caching), AsyncStorage, SQLite, Fastlane, Firebase Cloud Messaging (Push Notifications), Jest + React Native Testing Library, ESLint + Prettier, Metro Bundler, Fastlane + Firebase App Distribution, GitHub Actions (CI/CD), Bitrise (optional), REST APIs, GraphQL (Apollo Client)

Senior iOS App Developer Allstate Insurance, Atlanta, GA

Sep 2017 – Oct 2019

Link: <https://apps.apple.com/us/app/allstate-mobile/id364376344>

Summary: Senior iOS App Developer on the Allstate Mobile app, I contributed to core insurance workflows including Policy Management, Bill Payment, Claims Filing, and Digital ID Card Access. I helped modernize legacy Objective-C modules to Swift, introduced MVVM + Coordinator patterns, and improved app performance and security. Focused on reliability, accessibility, and seamless customer experiences for millions of users nationwide.

- Developed the Policy Management module, including Policy Details, Coverage Breakdown, and Digital ID Card screens using UIKit + MVVM, improving load time and UI consistency.
- Enhanced the Billing & Payment flow using Swift + URLSession (closures) for network calls, streamlining bill payment reliability and reducing transaction failures by 18%.
- Implemented the Claims Filing workflow, integrating AVFoundation for photo/video capture and Core Data for offline claim drafts, enabling users to submit claims even without connectivity.
- Built the Roadside Assistance and Accident Support modules with MapKit + CoreLocation, allowing users to share exact coordinates for towing or emergency support.

- Refactored Objective-C components into modular Swift frameworks for Policy, Billing, and Claims, improving maintainability and enabling faster iteration on new features.
- Implemented SSL Pinning and AES-256 encryption for network and local data security, aligning with Allstate's enterprise compliance standards.
- Improved performance on the Dashboard and My Account screens using NSCache and background prefetching, reducing loading time by 20%.
- Enhanced accessibility by implementing VoiceOver, Dynamic Type, and UIAccessibility APIs, ensuring compliance with WCAG 2.0 AA standards.
- Established CI/CD pipelines using Fastlane and Jenkins, automating build, test, and release workflows, and improving delivery consistency.
- Participated in code reviews and architecture discussions, enforcing Swift coding standards and MVVM best practices.
- Worked closely with design teams to revamp the Home Dashboard, Policy Summary, and Payment Due widgets for improved usability and clarity.
- Built user interfaces with Storyboards and AutoLayout, utilizing size classes for compatibility across different screen sizes.
- Worked on Grand Central Dispatch (GCD) to build a responsive UI and implemented background threads.
- Prepared the documentation of the applications in which I was involved and test case documentation with sample inputs.
- Built iOS Storyboards and established IBOutlet, IBActions, and created and performed segues between different scenes in the storyboard
- Used debugging tools like Instruments, Crash Logs, LLDB, NSLog, and Breakpoints to troubleshoot and resolve QA-reported bugs.
- Mentored junior developers on UIKit layouts, networking patterns, and secure data handling practices.
- Demonstrated proficiency in Swift, Objective-C, and key iOS frameworks like Core Data and UIKit.
- Maintained a strong commitment to organizational goals and teamwork, fostering a productive work environment.

Tech stack: Swift, Objective-C, UIKit, MVVM, Coordinator, Core Data, Keychain, URLSession, RESTful APIs, JSON, AVFoundation, MapKit, CoreLocation, Auto Layout, NSCache, CocoaPods, Fastlane, XCTest, XCUITest, Instruments, SSL Pinning, AES Encryption, LocalAuthentication, VoiceOver, Dynamic Type, UIAccessibility, GitHub, Agile, Bitrise

iOS Developer

Jun 2015 – Aug 2017

Regeneron Pharmaceuticals, Indianapolis, IN

Link: <https://apps.apple.com/us/app/myway-patient-support/id6443848439>

Summary: As a Senior iOS Engineer on the **MyWay Patient Support** app, I developed and enhanced key patient workflows including enrolment, insurance verification, treatment tracking, and medication reminders. The app was built with UIKit and Objective-C, with newer modules transitioned to Swift. Focused on HIPAA-compliant data handling, offline reliability, and secure authentication for healthcare users in field and remote conditions.

- Developed the Patient Dashboard screen using UIKit + MVVM, displaying current treatment status, upcoming appointments, and reminders, improving engagement with adherence tracking by 20%.
- Built the Treatment Reminder and Medication Tracker modules, integrating Core Data for offline storage and ULocalNotification for reminders, ensuring continuity of care even when offline.
- Engineered the Patient Enrollment and Insurance Verification screens using Alamofire + SwiftyJSON for RESTful API integration, enabling smooth registration and claim validation workflows.
- Implemented Push Notifications for appointment reminders, refill alerts, and care-plan updates via UserNotifications and APNS, boosting adherence to medication schedules.
- Integrated Keychain Services for secure storage of login tokens and sensitive patient identifiers, aligning with HIPAA and 21 CFR Part 11 standards.
- Revamped the Claims Submission and Messaging Center screens, introducing asynchronous API handling using NSURLSession completion handlers, reducing API timeouts and retries.

- Optimized performance across Treatment Summary, Claims, and Insurance Detail screens using lazy image loading and background fetch, reducing memory usage by 15%.
- Enhanced accessibility with VoiceOver, Dynamic Type, and UIAccessibility APIs across onboarding and reminder screens to meet WCAG 2.0 AA compliance.
- Refactored networking code from NSURLConnection to NSURLSession, standardizing request handling and improving reliability on poor network connections.
- Streamlined build and release workflows using Fastlane + Jenkins, automating builds, test runs, and TestFlight deployment to accelerate FDA audit cycles.
- Conducted performance profiling with Xcode Instruments (Time Profiler, Allocations) to identify and fix leaks on Dashboard and Claims modules.
- Collaborated cross-functionally with backend engineers, QA, and UX designers to ensure secure, compliant, and user-friendly experiences for patients and caregivers.
- Mentored junior developers on Swift best practices, UIKit view optimization, and secure data persistence techniques for healthcare-grade apps.

Tech Stacks Swift, Objective-C, UIKit, MVC/MVVM, Alamofire, SwiftyJSON, NSURLSession, Core Data, Keychain, APNS, UserNotifications, CocoaPods, Auto Layout, Core Animation, Core Graphics, Fastlane, Jenkins, XCTest, XCUITest, Instruments, RESTful APIs, JSON, VoiceOver, Dynamic Type, UIAccessibility, GitHub

iOS Mobile App Developer Rivian, Toledo, OH

Jan 2013 – May 2015

Link: <https://apps.apple.com/us/app/rivian/id1570215232>

Summary: As an iOS Developer for Rivian app, worked as an iOS Developer on a customer-facing mobile app, contributing to features such as user login, profile management, and push notifications. Built UI screens using UIKit and Storyboards, integrated REST APIs for real-time data, and implemented Core Data for offline support. I gained strong experience with Objective-C

- Built login and registration screens in UIKit with form validation and error handling.
- Integrated RESTful APIs for fetching user data and posting updates using NSURLSession.
- Used Core Data for caching user information and enabling offline access to recent activity.
- Implemented push notifications to alert users of account updates and new content.
- Assisted in Objective-C to Swift migration, rewriting smaller utility modules.
- Conducted basic unit tests with XCTest to validate login and profile features.
- Built the Vehicle Dashboard screen using UIKit, showing charging status, tire pressure, and battery levels in real time.
- Integrated Rest APIs to fetch vehicle data faster, cutting down response times and improving user experience.
- Added UI snapshot tests with XCUITest to make sure important screens like Charging and Remote Commands looked consistent after updates.
- Worked closely with the firmware and cloud teams to ensure remote commands (lock/unlock, climate control) synced reliably between the app and the vehicle.
- Implemented Keychain for secure storage of user login details, ensuring application security.
- Wrote an experimental RESTful API module for posting comments on listings.
- Leveraged GitLab for version control and collaborative development, ensuring a streamlined and organized workflow.

Tech Stack: UIKit, Storyboards, MVVM, Core Image, Sketch, XCTest, Jenkins, JIRA, Keychain, RESTful APIs, Github.

Education

Master's in Computer Science

Bachelor's in Computer Science

San Francisco Bay University, California, USA